```
Scenario Outline: Earning extra points on flights by Frequent Flyer status
  Given I am a <status> Frequent Flyer member
  When I fly on a flight that is worth <base> base points
  Then I should earn a status bonus of <bonus>
  And I should have guaranteed minimum earned points per trip of <minimum>
  And I should earn <total> points in all
Examples:
  | status   | base | bonus | minimum | total | notes
  | Standard | 439  | 0     | 0       | 439   |
  | Silver   | 439  | 220   | 500     | 659   | 50% bonus
  | Silver   | 148  | 74    | 500     | 500   | minimum points
  | Gold     | 474  | 400   | 1000    | 1000  | minimum points
  | Gold     | 2041 | 1531  | 1000    | 3572  | 75% bonus
```

Create new Frequent Flyer member with a given status ❶

Although this has a similar format to the embedded data tables in the previous section, it's a very different beast and generally easier to use. For example, in JBehave, for line ❶ you could simply reuse the step definition you saw earlier:

```
@Given("I am a $status Frequent Flyer member")
public void defineAMemberWithStatus(String status) {
    member = members.getMember().withStatus(status);
}
```

This is useful, as steps are often used interchangeably between the two types of scenarios.

6.2.7 Understanding scenario outcomes

Traditional unit tests generally *pass* (green) or *fail* (red). But in BDD tests, several extra outcomes are possible. For example, scenarios can also be *pending* (when they haven't been started yet, or haven't yet been completely implemented).

When you run a scenario, each step will have its own outcome. The overall result of the scenario will depend on the outcomes of the different steps. If all of the steps succeed, then the scenario will succeed. If one of the steps fails, then the scenario will fail. And if any of the steps are incomplete or undefined, then the scenario as a whole will be reported as pending. Furthermore, when a step fails or is pending, there's no point running the subsequent steps because the test outcome is already compromised. In this case, any steps following the failed or pending step are marked as *skipped*.

The outcome of each step depends on a number of factors (see figure 6.4), including whether the step succeeded or not and the outcomes of previous steps.

If a BDD tool finds no matching definition for a step in a scenario, it will report this step as pending. It will also write a skeleton step definition to the console, which you can use as a starting point for your own step definition.

Any step that's implemented and doesn't fail in some way is considered successful. Because of this, if you have only partially implemented a step, it may be reported as successful even if the corresponding application code isn't complete. To avoid this, you sometimes need to mark a step as *work-in-progress*. Each tool provides its own way of doing this. In Ruby, for example, you can mark a partially implemented test as pending like this: